SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO1- AT2

NAME : PERAM VIVEKANANDA REDDY

REG NO : 192372330

COURSE : CSA1305 THEORY OF COMPUTATION

SIGN : 

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: Theory of Computation **COURSE CODE: CSA13**

COURSE OUTCOME COVERED

CO1: *Apply mathematical foundations such as formal proofs, induction, and basic concepts of automata theory to solve computational problems. (BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**
Duration : 1 Hour

Simulation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed, Opening, Open,* and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

3 Design DFA using simulator to accept the string the end with ab over set {a,b)
W= aaabab

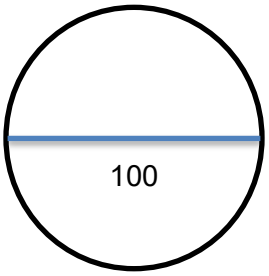
4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any

combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RUBRICS FOR CALCULATION



Question 1: Design of a Non-Deterministic Finite Automaton (NFA) for an Automatic Door System

Aim

To design and simulate a **Non-Deterministic Finite Automaton (NFA)** for an Automatic Door System based on sensor inputs using AutoSim.

Procedure

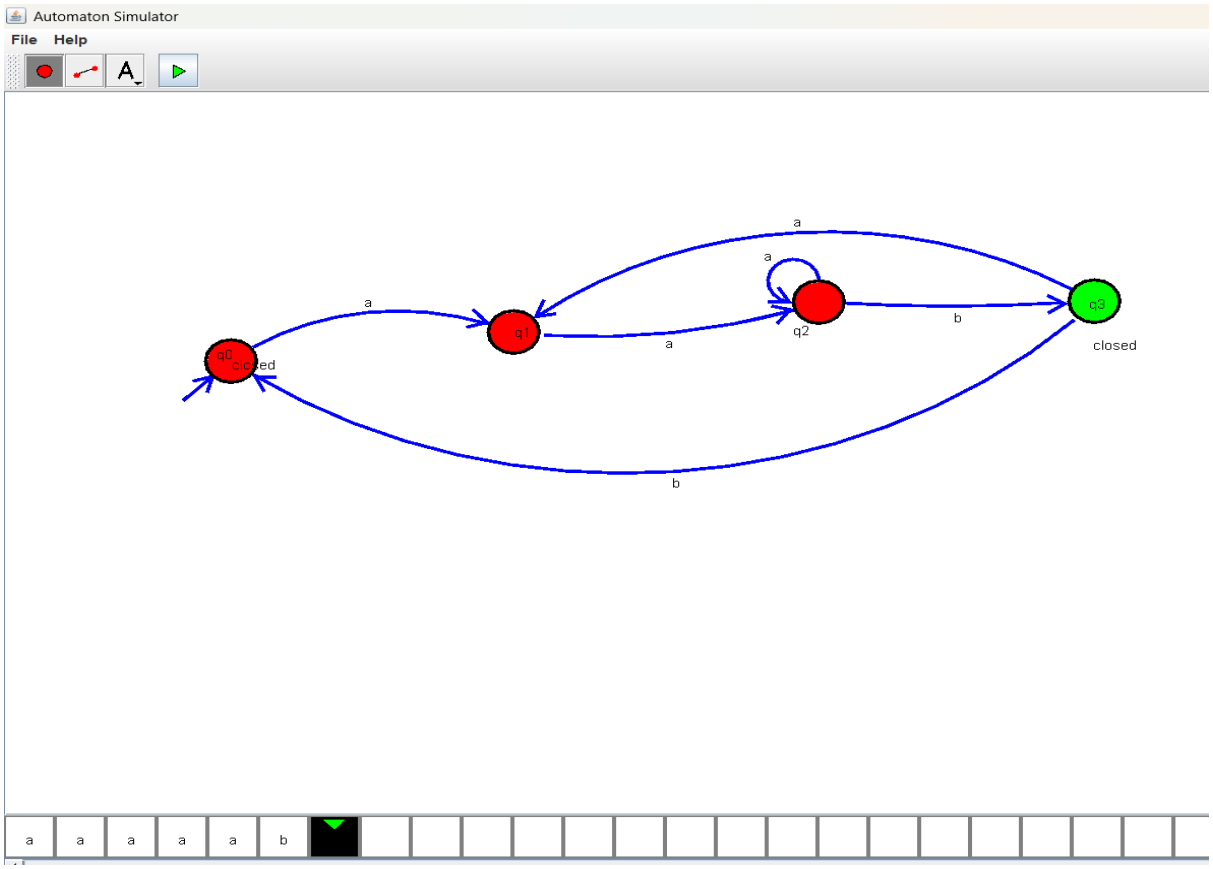
1. Open the **AutoSim** software.
2. Create four states:
    - q0 – Closed (Initial State)
    - q1 – Opening
    - q2 – Open
    - q3 – Closing
3. Set q0 as the **initial state**.
4. Set q0 as the **final (accepting) state** since the door finally returns to the closed position.
5. Use the input alphabet:
    - P = Person detected
    - N = No person detected
6. Add the following transitions:

From State	Input	To State
q0	a	q1
q1	a	q2
q1	b	q2
q2	a	q2
q2	b	q2
q2	b	q3
q3	b	q0
q3	a	q1

7. Save the automaton.
8. Click **Run/Simulate**.

9. Enter test input strings such as PP, PPN, or PPNN and verify the state transitions.
10. Observe the output and verify that the NFA behaves as expected.

Output:



Result

Thus, the **NFA for the Automatic Door System** was successfully designed and verified using AutoSim.

Question 2: Design of a DFA for a Traffic Light Control System

Aim

To design and simulate a **Deterministic Finite Automaton (DFA)** for a Traffic Light Control System using AutoSim, representing the sequence of traffic signals (Red → Green → Yellow → Red).

Procedure

1. Open the **AutoSim** software.
2. Create three states:
    - q0 – Red
    - q1 – Green

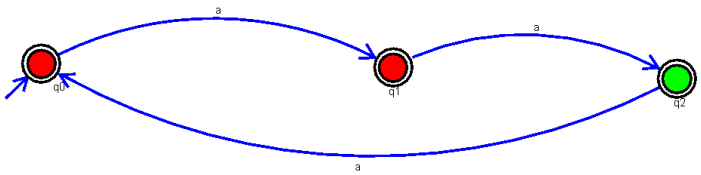
- q2 – Yellow

3. Set q0 as the **initial state**.
4. Since the traffic light operates continuously, all states (q0, q1, q2) may be treated as accepting states (or as required by your simulator).
5. Use the input symbol:
   - T = Timer expires (or timer signal)
6. Add the following transitions:

From State	Input	To State
q0	a	q1
q1	a	q2
q2	a	q0

7. Save the DFA.
8. Click **Run/Simulate**.
9. Enter the input string **aaa** (or multiple as such as aaaaa) to observe the cyclic transitions.
10. Verify that the traffic light changes in the order **Red → Green → Yellow → Red** repeatedly.

OUTPUT



Result

Thus, the **DFA for the Traffic Light Control System** was successfully designed and verified using AutoSim. The automaton correctly implements the sequence **Red → Green → Yellow → Red** in a continuous cycle.

Experiment 3: Design a DFA to Accept Strings Ending with "ab" over the Alphabet {a, b}

Aim

To design and simulate a **Deterministic Finite Automaton (DFA)** using AutoSim that accepts all strings ending with **"ab"** over the alphabet **{a, b}**.

Alphabet

$$\Sigma = \{a, b\}$$

States

- **q0** – Initial state
- **q1** – Last symbol read is **a**
- **q2** – String ends with **ab** (Final State)

Initial State

q0

Final State

q2

Transition Table

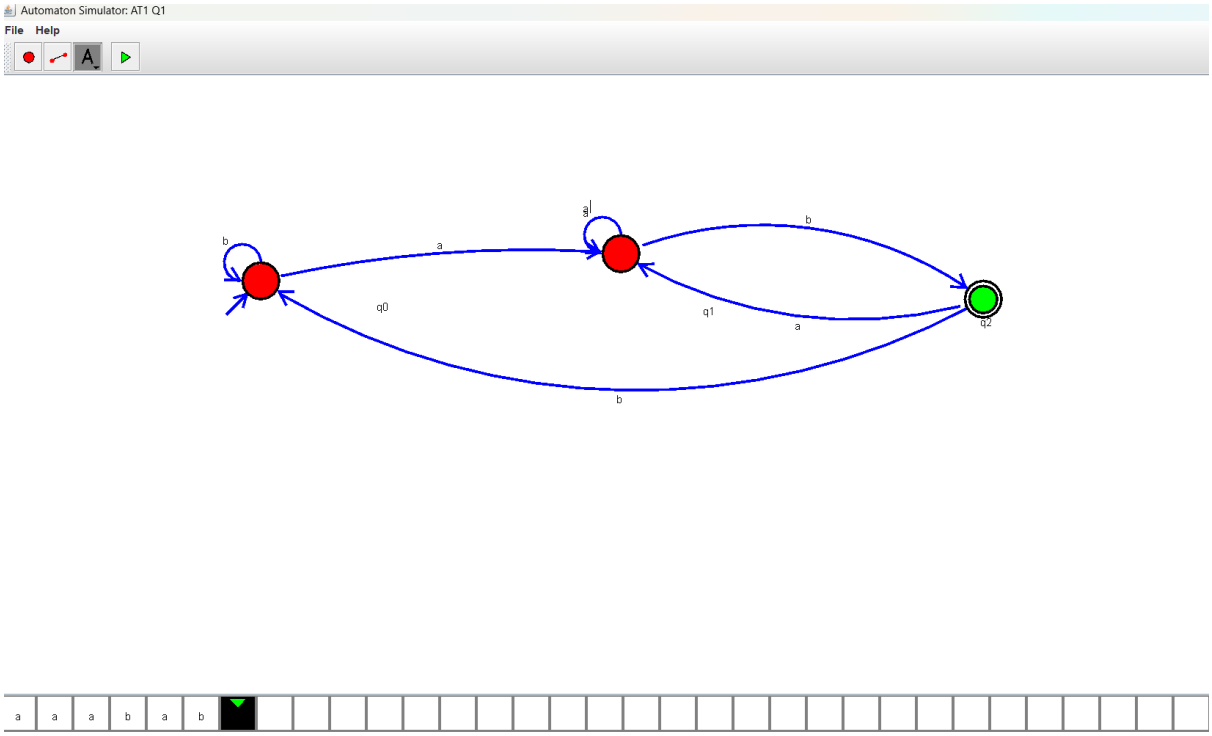
Current State	Input a	Input b
q0	q1	q0
q1	q1	q2
q2	q1	q0

Procedure

1. Open the **AutoSim** software.
2. Create three states: **q0**, **q1**, and **q2**.
3. Set **q0** as the **initial state**.
4. Set **q2** as the **final (accepting) state**.
5. Use the input alphabet **{a, b}**.

6. Add the transitions according to the transition table.
7. Save the DFA.
8. Click **Run/Simulation**.
9. Enter input strings such as **ab**, **aaab**, **bab**, and **aaabab**.
10. Verify that only strings ending with **"ab"** are accepted.

Output:



Result

Thus, the **Deterministic Finite Automaton (DFA)** to accept all strings ending with **"ab"** over the alphabet **{a, b}** was successfully designed and verified using AutoSim.

Experiment 4: Design a DFA to Accept Only the Strings "c", "bc", and "bcaaa"

Aim

To design and simulate a **Deterministic Finite Automaton (DFA)** using AutoSim that accepts only the strings **"c"**, **"bc"**, and **"bcaaa"** over the alphabet **{a, b, c}**.

Alphabet

$$\Sigma = \{a, b, c\}$$

States

- **q0** – Initial state
- **q1** – After reading **b**
- **q2** – After reading **c** (Final state for **"c"** and **"bc"**)
- **q3** – After reading **bca**
- **q4** – After reading **bcaa**
- **q5** – After reading **bcaaa** (Final state)
- **qd** – Dead state

Initial State

q0

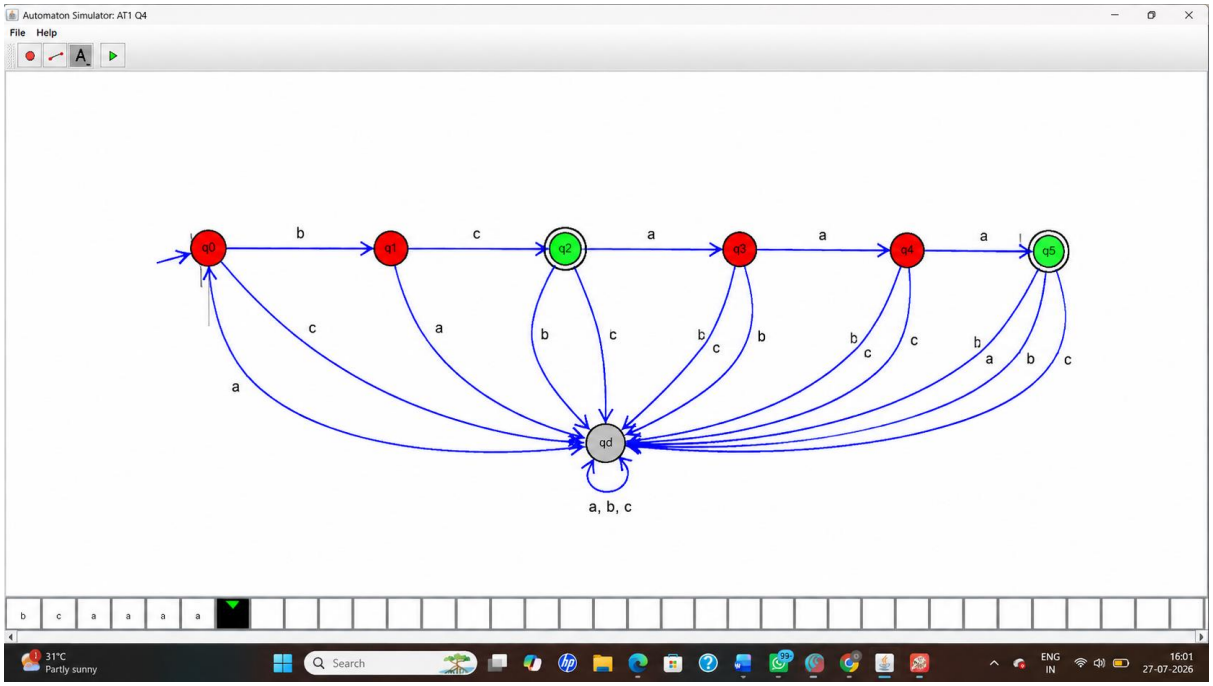
Final States

q2, q5

Transition Table

Current State	**a**	**b**	**c**
q0	qd	q1	q2
q1	qd	qd	q2
q2	q3	qd	qd
q3	q4	qd	qd
q4	q5	qd	qd
q5	qd	qd	qd
qd	qd	qd	qd

Output:



Result

Thus, the Deterministic Finite Automaton (DFA) to accept only the strings "c", "bc", and "bcaaa" over the alphabet {a, b, c} was successfully designed and verified using AutoSim.

Experiment 5: Design a DFA to Accept All Strings Beginning with 'a' or 'b'

Aim

To design and simulate a **Deterministic Finite Automaton (DFA)** using AutoSim that accepts all **non-empty strings** over the alphabet **{a, b}** which begin with either **'a'** or **'b'**.

Alphabet

$$\Sigma = \{a, b\}$$

States

- **q0** – Initial state
- **q1** – Accepting (Final) state

Initial State

q0

Final State

q1

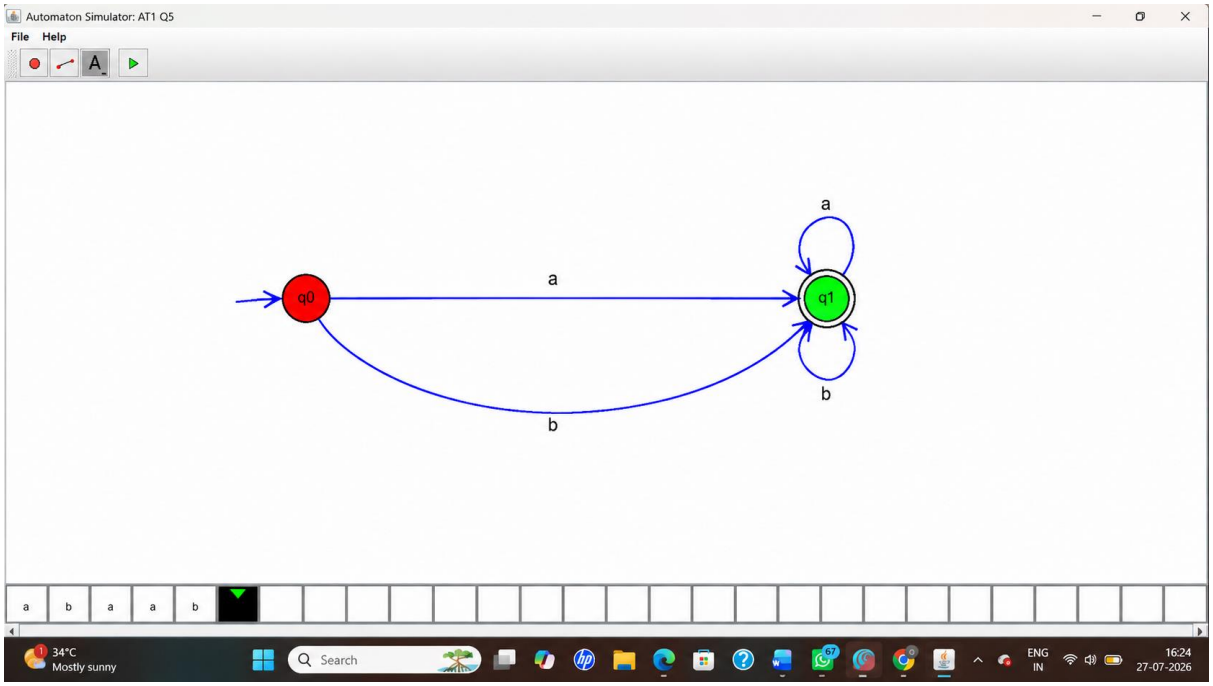
Transition Table

Current State	Input a	Input b
q0	q1	q1
q1	q1	q1

Procedure

1. Open the **AutoSim** software.
2. Create two states: **q0** and **q1**.
3. Set **q0** as the **initial state**.
4. Set **q1** as the **final (accepting) state**.
5. Use the input alphabet **{a, b}**.
6. Add the following transitions:
    - q0 → q1 on input **a**
    - q0 → q1 on input **b**
    - q1 → q1 on input **a**
    - q1 → q1 on input **b**
7. Save the DFA.
8. Click **Run/Simulation**.
9. Test the automaton with different input strings such as **a**, **b**, **ab**, **ba**, **aab**, and **bba**.
10. Verify that all non-empty strings beginning with **a** or **b** are accepted.

DFA Diagram :



Sample Input and Output

Input String	**Result**
a	Accepted
b	Accepted
ab	Accepted
ba	Accepted
aab	Accepted
bba	Accepted
ε (empty string)	Rejected

Result

Thus, the **Deterministic Finite Automaton (DFA)** to accept all non-empty strings over the alphabet **{a, b}** beginning with **'a'** or **'b'** was successfully designed and verified using AutoSim.